No GIL, Real Gains

逐页学习手册

PyData PyCon Yerevan 2026 · 35 页 deck 的完整注释版。这份手册按 deck 页码逐页展开：每页先给缩略图，再解释这页在讲什么、每个术语什么意思、每个数字从哪来。GIL 移除不是我们做的工作，但要讲好这场 talk，需要理解 CPython 这三十年在这件事上的来龙去脉——这份手册就是为此准备的。

更新日期：2026-07-23（v3）· 配套文件：slides/chdb-nogil-pydata-yerevan-2026.html（35 页版，2026-07-23 修订：#128/#131 修复落地）

本版（v3）更新：chdb-core PR #141/#142 已合并，两个 rc bug 都修了

- **第 24 页（This is what no-GIL was made for）**：stock/3.14t 对比 bullet 现在带修复后的实测数字（stock 平在 ~0.85 s；3.14t 0.72 → 0.15 s，32 线程不退化）；per-row boxing 条目更新为 vectorcall 优化 4.7×（#142）。
- **第 29 页（What broke）第 ⑤ 条**：改为已修复 + 确诊根因（GC stop-the-world × import-cache call_once 死锁），新教训 “detach before you block”。
- **第 30 页脚注**：“Known sharp edges” 改为 “Both rc sharp edges are fixed for 26.5.1”，附 #128 根因一句话，金句 “That's what benchmarks are for” 回归。
- 附录 Q&A 的 #128/#131 条目按确诊结论重写——注意：issue 里原来的三个假设都不是主因。

第 0 部分 · 5 分钟预备知识 读懂全部 deck 的最小背景

GIL 到底是什么

GIL = Global Interpreter Lock，全局解释器锁。CPython 里的一把全局互斥锁：任何线程要执行 Python 字节码、或调用绝大多数 C API，必须先拿到它。任意时刻只有一个线程在“跑 Python”。线程做阻塞 I/O 时会临时放掉 GIL，所以多线程做网络/磁盘 I/O 一直是有效的；无效的是多线程跑 CPU 密集的 Python 代码——16 个线程也只有一个核在干活。

GIL 为什么存在：引用计数

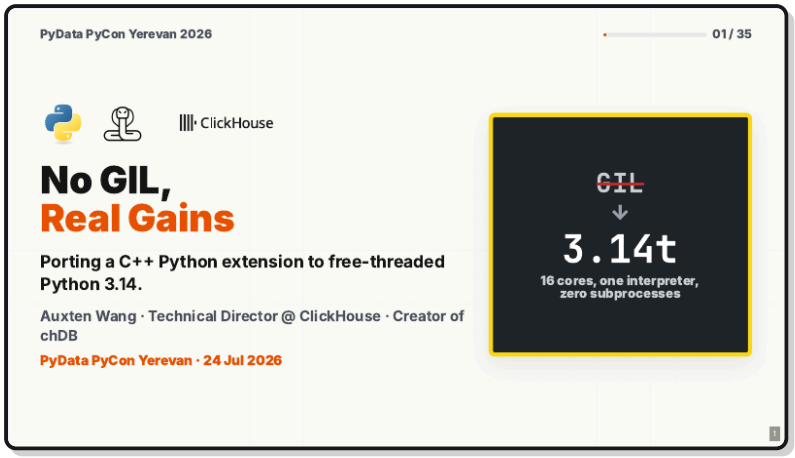
CPython 用引用计数（reference counting）管理内存：每个对象头部有个 `ob_refcnt` 字段，多一个引用就 +1（`Py_INCREF`），少一个就 −1（`Py_DECREF`），归零立即释放。这个 +1/−1 是普通的、非原子的内存操作——两个线程同时对同一对象加减，计数就会错乱，导致提前释放（崩溃）或永不释放（泄漏）。GIL 一锁了之：既然同一时刻只有一个线程在跑，计数天然安全。副作用是整个 C 扩展生态都写成了“默认单线程”的样子——这正是本 talk 的主题：把这个假设拆掉之后会发生什么。

本手册反复出现的几个词

- **free-threading / no-GIL / 3.14t**：同一件事的三种说法：没有 GIL 的 CPython 构建。`t` 后缀 = free-threaded，是一个独立的解释器二进制（`python3.14t`），和普通 3.14 并存。
- **wheel / cp314t**：wheel 是 Python 的二进制安装包格式。文件名里的 `cp314` 表示“给 CPython 3.14 编译的”，`cp314t` 则是给 free-threaded 3.14 编译的——两者 ABI 不同，不能混用。一个库“支持 free-threading”的硬指标就是它在 PyPI 上发没发 t 后缀的 wheel。
- **stable ABI / abi3 / Limited API**：正常情况下 C 扩展要对每个 Python 版本各编一份。CPython 提供一个“受限 API”子集（Limited API），只用它编出来的扩展叫 abi3 wheel，一份二进制跑 3.9~3.14 所有版本。代价是很多高性能内部接口不让用。free-threaded 构建目前不支持 stable ABI——这在 deck 后半段是个重要情节。
- **PEP**：Python Enhancement Proposal，Python 的设计提案文档。本 deck 涉及：PEP 703（把 GIL 做成可选，2023 年接受）、PEP 779（宣布 free-threading 转正为官方支持，3.14）、PEP 683（不朽对象）。
- **cache line（缓存行）**：CPU 缓存的最小单位（通常 64 字节）。多个核写同一个缓存行时，硬件一致性协议（MESI）会让缓存行在核间来回“弹跳”，每次弹跳耗上百个时钟周期。理解了这个，就理解了 2016 年 Gilectomy 为什么慢 19 倍。

第 1 部分 · 逐页注释

第 1 页 标题页 · No GIL, Real Gains



这页在讲什么：talk 的一句话主张——把一个 C++ 写的 Python 扩展（chDB）移植到 free-threaded Python 3.14，得到了真实的性能收益。

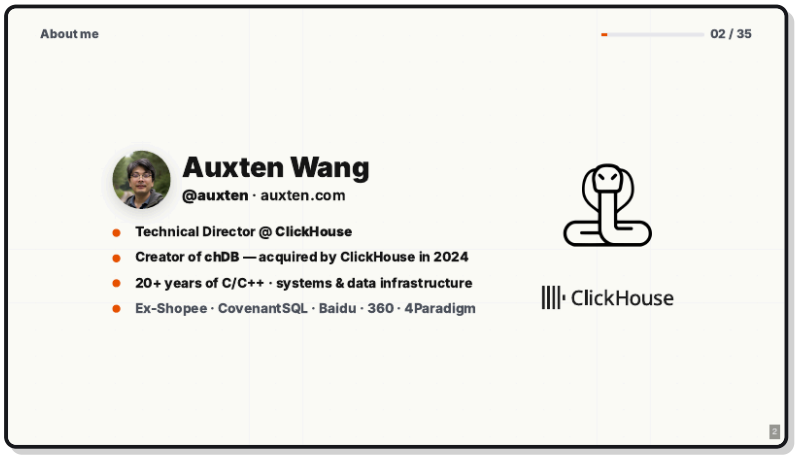
Porting a C++ Python extension

“Python 扩展”= 用 C/C++/Rust 写、编译成动态库、供 Python import 的模块（NumPy、pandas 的核心都是）。chDB 就是这样一个扩展：外面是 Python API，里面是 ClickHouse 的 C++ 引擎。

右侧黑卡：GIL（划掉）→ 3.14t →“16 cores, one interpreter, zero subprocesses”

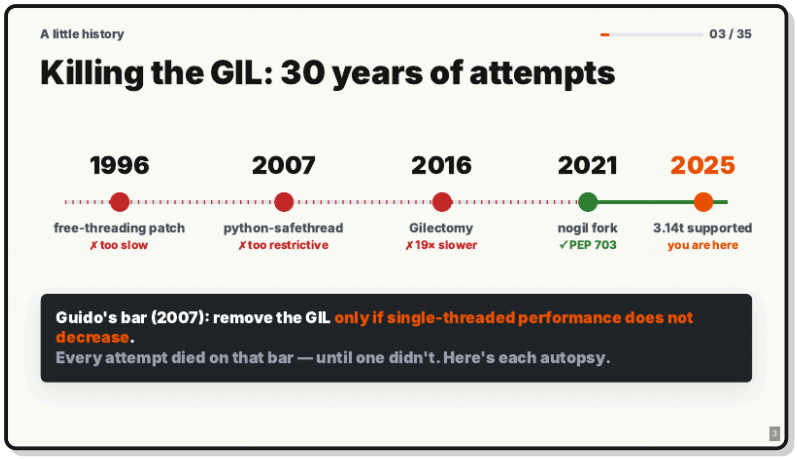
全 deck 的电梯陈述：过去用满 16 核要开 16 个进程（multiprocessing），现在一个解释器进程里的 16 条线程就能各占一核。“zero subprocesses”是对旧世界（fork + pickle）的点名。

第 2 页 自我介绍 · About me



这页在讲什么：建立讲者可信度：chDB 作者、被 ClickHouse 收购、20 年 C/C++ 系统经验。讲的时候一句带过即可，重点是让观众记住“chDB = 嵌入式 ClickHouse”这一个事实。

第 3 页 历史时间轴 · Killing the GIL: 30 years of attempts



这页在讲什么：移除 GIL 不是新想法——1996 年起至少四次大尝试，前三次全部失败，直到 2021 年。时间轴上红色虚线段=失败年代，绿色实线=成功路径，2025 的橙点=3.14t 转正（“you are here”）。

Guido's bar（Guido 的验收线）

Guido van Rossum（Python 之父）2007 年在 “It isn't Easy to Remove the GIL” 一文中立下的规矩：欢迎任何移除 GIL 的补丁，前提是单线程程序不能变慢（对现有 C 扩展的破坏也要可控）。这条线是理解前三次失败的钥匙——它们都不是“做不出来”，而是“做出来了但单线程变慢，所以不能合入”。绝大多数 Python 程序是单线程的，为了少数并行场景让所有人变慢，不可接受。

为什么讲历史

听众里 90% 没研究过 GIL。四次失败各自死于一个具体技术原因，把这四个原因讲清楚，观众就自动理解了“为什么这问题难”以及“2021 年到底改变了什么”。这比直接堆砌 3.14t 的新机制有效得多。

第 4 页 1996：第一次尝试 · Greg Stein's free-threading patch



这页在讲什么：1996 年（Python 1.4 时代！）Greg Stein 就做出了完整的 GIL 移除补丁——思路是“一把大锁换成每个可变结构一把小锁”。右侧的复古终端卡片是风格化的“史料”：还原他在 comp.lang.python 上发布补丁的场景。

fine-grained locks（细粒度锁）

与“一把全局锁”（coarse-grained）相对：给 dict、list、module 等每个共享结构单独上锁。并发度高了，但每次操作都要过锁——加锁/解锁本身有开销，单线程程序为不存在的并发白白付费。

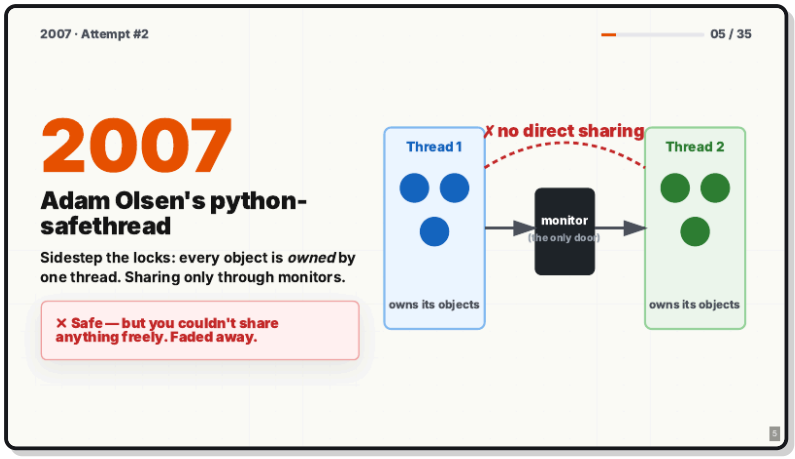
~2× slower single-threaded → shelved

补丁让单线程 Python 慢了约一倍（不同基准 1.6~2 倍不等，Larry Hastings 2016 年回顾时引用的数字更高）。撞上 Guido 那条线 → 束之高阁。

PyThreadState survived（幸存者）

这次尝试留下的遗产：`PyThreadState`——CPython 里“每线程一份”的解释器状态结构，今天仍在。第 26 页 war story 里崩溃的直接原因就是“线程没有 thread state”，伏笔在这里埋下。

第 5 页 2007：第二次尝试 · Adam Olsen's python-safethread



这页在讲什么：换思路：不拆锁，改内存模型。每个对象归创建它的线程所有（ownership），跨线程共享必须走受控通道（monitor）。示意图：两个线程各自的对象圈在自己领地里，中间黑色 monitor 是唯一的门，顶上红色虚线=禁止直接共享。

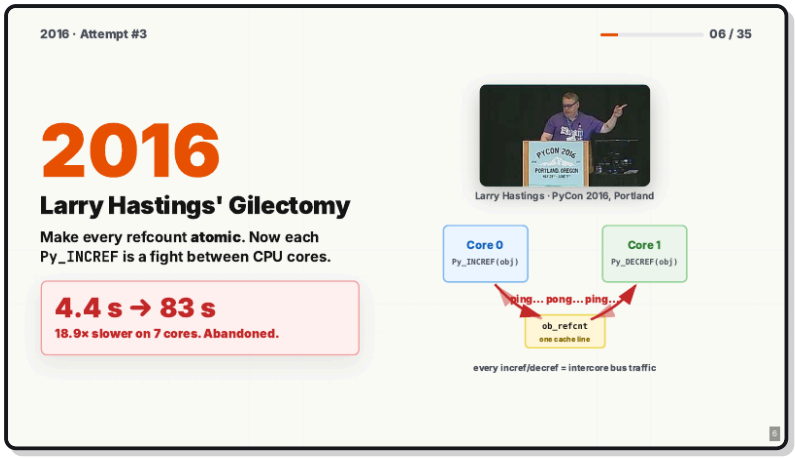
monitor（监视器）

并发编程的经典构造：把数据和保护它的锁绑在一起，所有访问必须经过它。可以类比今天 Rust 的 ownership/borrow 或 actor 模型——思想是先进的。

为什么“太受限”

Python 生态几十年的代码都假设“任何对象随手传给任何线程”。safethread 要求重写这种习惯，兼容性代价太大，社区不跟 → 项目自然消亡。教训：技术可行 ≠ 生态可行。这条教训在第 16–17 页（生态适配进度）会再次出现。

第 6 页 2016：第三次尝试 · Larry Hastings' Gilectomy



这页在讲什么：全 deck 技术上最有戏剧性的失败。Gilectomy（“GIL 切除术”，仿外科手术命名）把每个对象的引用计数改成原子操作——结果 7 核并行反而比单核慢 18.9 倍。照片是 Larry 在 PyCon 2016 讲台上（视频截帧）。

atomic（原子操作）

CPU 保证“读–改–写”三步不被其他核插队的指令（如 `lock inc`）。看似只是把普通 +1 换成原子 +1，但原子写要求当前核独占那个缓存行。

cache line ping-pong（缓存行乒乓）——右图的核心

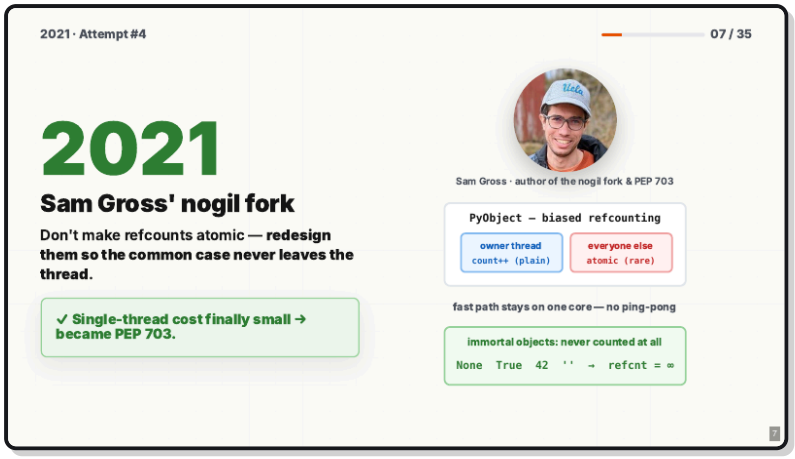
Python 程序无时无刻不在对热门对象做 INCREF/DECREF（传参、赋值、进出作用域都会触发）。多个核同时操作同一对象的 `ob_refcnt`，那 64 字节的缓存行就在核间来回弹：Core 0 写一次要把行从 Core 1 抢过来，Core 1 再写又抢回去……每次弹跳走核间总线，耗几百周期。并行度越高，弹得越凶，越慢。

4.4 s → 83 s（18.9×，7 核）

Larry 2017 年 6 月自己公布的测量（fib 基准）：普通 CPython 4.4 秒，Gilectomy 用 7 个核跑 83 秒。他后来又试了 buffered reference counting（refcount 记账事务日志）也没能救回来，项目 2017 年后放弃。数字出处：LWN《Progress on the Gilectomy》及 Victor Stinner 的回顾文章。

> **本页的教训（承上启下）：refcounting 是 GIL 移除的真正 boss。锁好拆，计数难改——这直接解释了第 7 页 nogil 为什么把全部聪明才智花在“重新设计引用计数”上。**

第 7 页 2021：成功的一次 · Sam Gross' nogil fork



这页在讲什么：Meta 工程师 Sam Gross（照片，GitHub @colesbury）在 Python 3.9 上做出 nogil fork：不硬扛原子操作，而是重新设计引用计数，让最常见的情况根本不跨核。单线程开销终于压到可接受 → 2023 年被接受为 PEP 703。

biased refcounting（偏向引用计数）——右图上半

出自 Jiho Choi 等 2018 年的论文。每个对象存两个计数器：owner 线程（创建它的线程）用普通非原子加减——快，不弹缓存行；其他线程用另一个原子计数器。关键洞察：绝大多数对象一生只被一个线程碰，走快路径；跨线程共享是少数，才付原子的代价。对象销毁时两个计数器合并结算。“偏向”= 偏向 owner。

immortal objects（不朽对象，PEP 683）——右图下半

`None`、`True` / `False`、小整数、驻留字符串这些“全民对象”被所有线程高频引用，是缓存行弹跳的最大源头。解法干脆：把它们的 refcnt 设成特殊值（视作 ∞），INCREF/DECREF 变成空操作。永远不写 → 缓存行永远 Shared 状态 → 零争用。反正这些对象也永远不会被释放。

图上没画、但注释该知道的另外两件“配料”

① deferred reference counting：函数、模块、代码对象这类长命对象，解释器循环里跳过计数，GC 时再校正；② mimalloc：微软的线程安全内存分配器，替换 CPython 的 pymalloc，还让 GC 能不依赖全局链表找到所有对象。第 14 页“黑魔法”会把全套配方展开。

第 8 页 落地时间线 · The one that landed: free-threaded Python



这页在讲什么：从 fork 到官方：PEP 703（3.13，2024，实验性）→ PEP 779（3.14，2025，正式支持）→ Phase III（未来：变成默认构建）。

PEP 703 · 3.13 · “~40% penalty”

3.13 的实验性 free-threaded 构建慢约 40%，主因是 specializing adaptive interpreter（3.11 引入的字节码自适应加速，PEP 659）在 FT 构建里被整体关闭——它的内联缓存不是线程安全的。

PEP 779 · 3.14 · “penalty 5–10%”

3.14 把 specialization 改造成线程安全后重新启用，官方口径的单线程开销降到约 5–10%。“officially supported”的意思：不再随时可能被砍，发行版/CI/云厂商可以放心投入。

opt-in build / cp314t

free-threading 目前是单独的解释器二进制（`python3.14t`），普通 `python3.14` 完全不受影响。代码示例里 `sys._is_gil_enabled()` 返回 False 表示“这个进程此刻真的没有 GIL”——这个函数在第 17、29 页还会出现，是贯穿全场的自检工具。

Phase III

PEP 703 定的三阶段路线的最后一步：实验性 → 官方支持（现在在这）→ 成为默认构建。“This train is leaving”的依据。

第 9 页 为什么重要 · Why it matters



这页在讲什么：把技术翻译成听众的利益。背景事实：单核频率十几年前就停止增长（功耗墙/Dennard scaling 终结），CPU 靠加核变宽——而 Python 一直只能用一根核。三张卡对应三类听众：

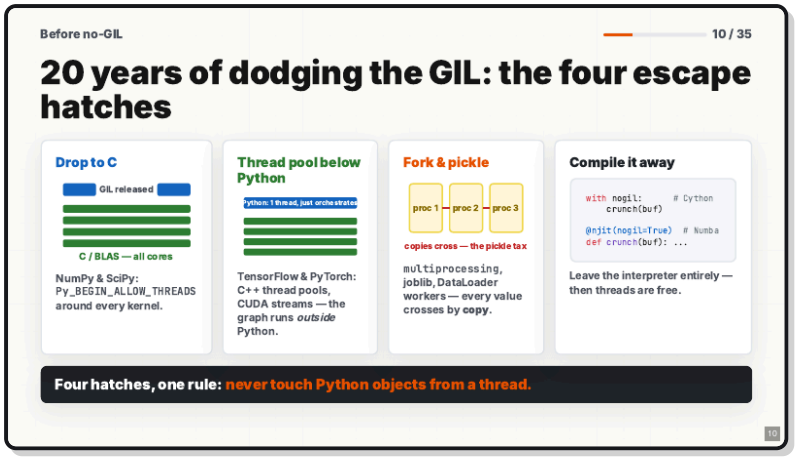
三张卡

- **Agent builders**：并行 subagent/工具调用共享同一份模型、缓存、向量索引——不用 pickle 不用 IPC（进程间通信）。对 2026 年的听众这是最戳的场景。
- **Desktop apps**：UI 线程保持响应 + 后台真并行，摆脱 multiprocessing 的 spawn/打包噩梦。
- **Servers & data tools**：“one copy of the model, not 16 workers × 16 copies”——预 fork 的 worker 模型内存是乘法，线程模型是共享。

黑卡结尾（新措辞）

“Same language, same process, 16× the compute — at a cost to extension authors.” 一句话完成转场：收益给用户，账单给扩展作者——正好接第 12 页。

第 10 页 GIL 时代的四个逃生舱 · 20 years of dodging the GIL



这页在讲什么：没有 no-GIL 的这些年，整个生态怎么活下来的——四种绕行套路。这页回答观众必然的疑问：“NumPy/PyTorch 不是早就多核了吗，那还要 no-GIL 干嘛？”（答案在下一页：能绕的都绕了，绕不掉的正是 Python 对象。）

① Drop to C（NumPy / SciPy 的路）

C 扩展在进入长时间纯 C 计算前调用 `Py_BEGIN_ALLOW_THREADS` 宏放掉 GIL，算完再拿回来。矩阵乘法期间 BLAS（底层线性代数库，OpenBLAS/MKL）可以开满核——前提是这段 C 代码完全不碰任何 Python 对象。小图：蓝色 Python 条断开的空档里，绿色 C 线程铺满。

② Thread pool below Python（TensorFlow / PyTorch 的路）

把并行整个做到 Python 之下：Python 只负责搭计算图/发指令（一根线程足矣），真正的执行在 C++ runtime 的线程池（Eigen、ATen）和 GPU 的 CUDA stream 里。Python 越“薄”越好。这解释了第 15 页 TensorFlow 为什么至今不急着适配 free-threading——它早就把 Python 架空了。

③ Fork & pickle（数据工程的日常）

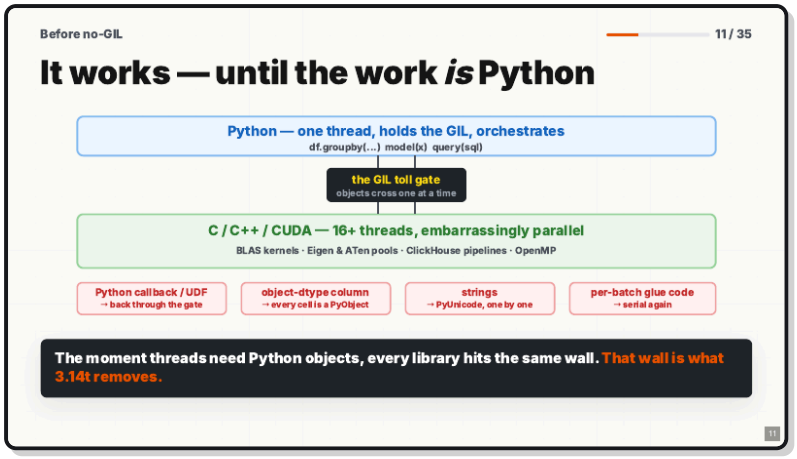
绕不过线程就开进程：`multiprocessing`、joblib（scikit-learn 并行后端）、PyTorch DataLoader 的 `num_workers`。每个进程有自己的解释器和 GIL，互不挡道。代价即“pickle tax”：进程不共享内存，每个跨进程的值都要序列化（pickle）→ 拷贝 → 反序列化；启动慢、内存翻倍、还有一堆不可 pickle 的对象（锁、连接、lambda）。

④ Compile it away（Cython / Numba 的路）

把热点函数编译成机器码并声明“这段不碰 Python 对象”：Cython 的 `with nogil:` 块、Numba 的 `@njit(nogil=True)`。编译后的代码不需要解释器，线程自由。限制同①：块内只能用 C 级类型。

底部黑卡是全页的钥匙：四个逃生舱共享同一条铁律——线程里绝不碰 Python 对象。碰一下（一次 INCREF、一次属性查找、一次装箱）就必须持有 GIL，并行瞬间归零。这条铁律就是下一页的“墙”。

第 11 页　套路的天花板 · It works — until the work is Python



这页在讲什么：把第 10 页的四条路抽象成一张总架构图，并指出它的失效条件。上层：Python 单线程编排（持 GIL）；下层：C/C++/CUDA 多线程狂奔；中间的黑色窄口=“GIL toll gate”（收费闸）——对象过闸，一次一个。

为什么“碰对象”必须过闸

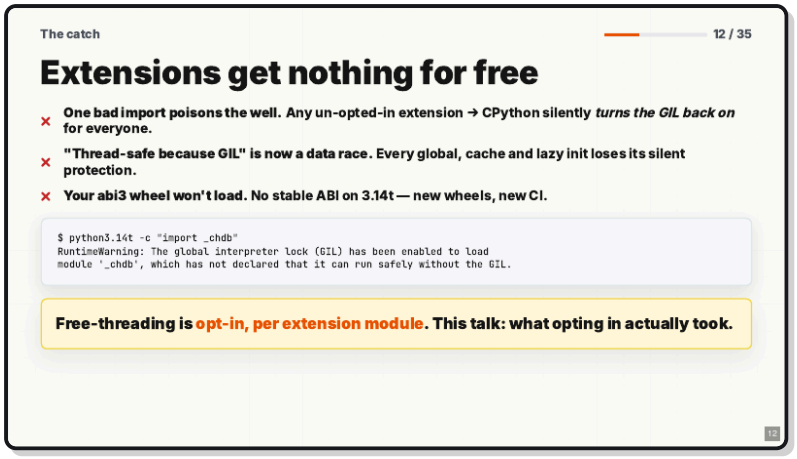
任何对 PyObject 的操作——读属性、装箱一个 int、创建一个 str、乃至仅仅 INCREF——都要求持有 GIL。所以只要下层线程需要 Python 世界的任何东西，就得排队过那道闸。

四个红色场景（从左到右）

① Python callback / UDF：C++ 引擎回调用户的 Python 函数（本 talk 的主线场景，第 22–24 页展开）；② object-dtype column：pandas/NumPy 的 `dtype=object` 列，每个单元格是一个 PyObject 指针——扫这种列 = 每行过一次闸；③ strings：Python 字符串是 PyUnicode 对象，逐个转换（第 27–28 页的主角）；④ per-batch glue：每个 batch 之间那点 Python 胶水代码，量不大但全是串行点（Amdahl 定律：串行部分决定加速上限）。

> **叙事枢纽：“每个库都撞上同一堵墙，3.14t 拆的就是这堵墙。”从这页起，deck 从“大家的故事”切到“chDB 的故事”——chDB 恰好四个场景全占（UDF、object 列、字符串、胶水），所以是绝佳案例。**

第 12 页 代价 · Extensions get nothing for free



这页在讲什么：转折页——3.14t 不是免费午餐，对扩展作者有三记闷棍。

① One bad import poisons the well（一颗老鼠屎）

机制：每个 C 扩展模块要显式声明“我能在无 GIL 下安全运行”。只要进程里 import 了任何一个没声明的扩展，CPython 就静默地把 GIL 重新打开——整个进程回到旧世界，只给你一行 RuntimeWarning（下方终端框就是实物：chDB 没声明时的真实输出）。这是保护机制（未声明的扩展在无 GIL 下可能直接崩），但意味着你的“no-GIL 部署”可能默默在跑 GIL。

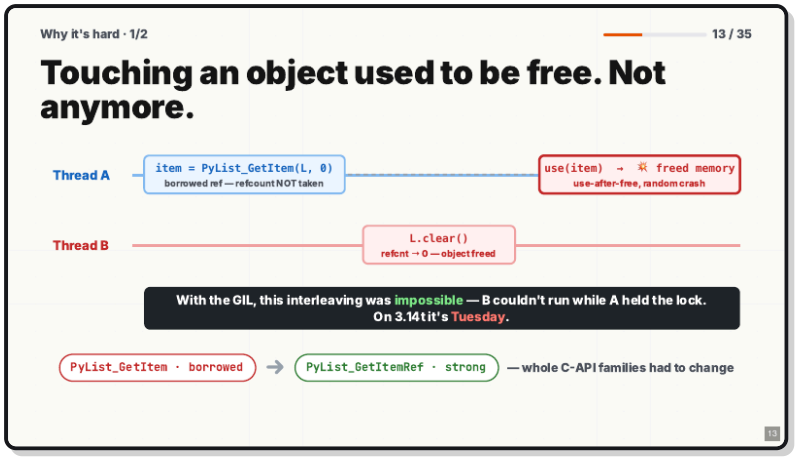
② “Thread-safe because GIL” is now a data race

过去 C 扩展里所有全局变量、缓存、懒初始化都躲在 GIL 的伞下——两个线程从不同时执行。伞一撤，这些全是数据竞争（data race：多线程无同步地读写同一内存，行为未定义）。第 25–26 页有 chDB 的实例。

③ Your abi3 wheel won't load

见预备知识“stable ABI”条：FT 构建不支持 Limited API，所以原来“一份 wheel 走天下”的发布策略作废，必须为 cp313t / cp314t 单独编译、单独跑 CI。

第 13 页 为什么难 1/2：借用引用 · Touching an object used to be free



这页在讲什么：用一张时序图讲清 free-threading 下最典型的一类崩溃：借用引用 + 并发释放 = use-after-free。这页是“库为什么都要重做对象读写”的最小可复现解释。

borrowed reference（借用引用）

C API 的老约定：`PyList_GetItem(L, 0)` 返回列表元素的指针，但不给引用计数 +1——你是“借”的，列表才是所有者。在 GIL 时代这很安全：你持着 GIL，别的线程根本没机会跑，更没机会把元素删掉。省一对 INCREF/DECREF，性能友好，所以借用引用遍布整个 C API 和无数扩展代码。

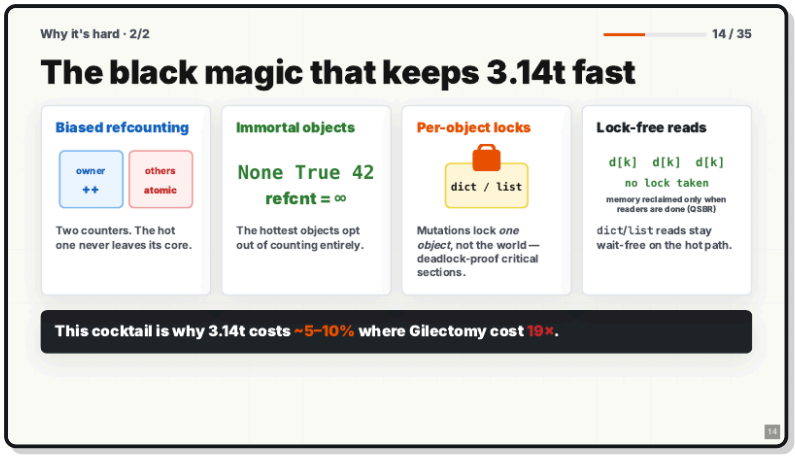
时序图逐帧

线程 A 借到 `item` → 线程 B（现在能同时跑了！）执行 `L.clear()`，元素 refcnt 归零、内存被释放 → A 再用 `item`：读已释放内存，随机崩溃或数据损坏。黑条点题：“GIL 时代这种交错不可能发生；3.14t 上这是家常便饭（Tuesday）。”

PyList_GetItem → PyList_GetItemRef

CPython 3.13 起为此增加的一族新 API（还有 `PyDict_GetItemRef` 等）：返回强引用（strong reference，计数 +1，用完自己 DECREF），内部用原子操作保证“取指针 + 加计数”不被并发释放插队。所以说“整族 C API 都得换”——每个用借用引用跨越“可能被并发修改”窗口的调用点都是雷。

第 14 页 为什么难 2/2：黑魔法四件套 · The black magic that keeps 3.14t fast



这页在讲什么：回答“Gilectomy 慢 19 倍，凭什么 3.14t 只慢 5–10%？”——四项核心机制，前两项在第 7 页出现过（这里是复习+集齐），后两项是新的。

① Biased refcounting

见第 7 页注释。一句话：热路径（owner 线程）的计数操作不出核，避开缓存行乒乓。

② Immortal objects

见第 7 页注释。一句话：最热的对象干脆不计数。

③ Per-object locks + critical sections（每对象锁）

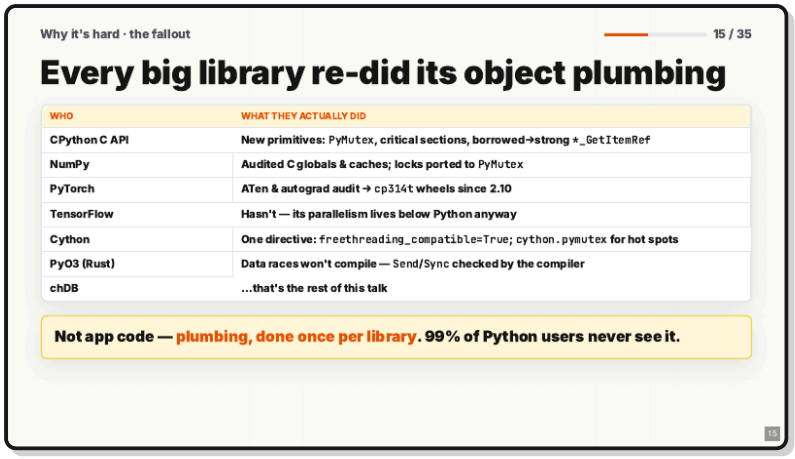
修改 dict/list 等容器时锁那一个对象，不锁世界。配套 API `Py_BEGIN_CRITICAL_SECTION(obj)` 有个精巧设计：当代码块内发生阻塞或再进解释器时，锁会被临时挂起再恢复——从机制上防死锁（deadlock-proof）。两个线程改两个不同的 dict 互不干扰。

④ Lock-free reads + QSBR

读 dict/list 完全不加锁（读比写频繁得多，读加锁就输了）。危险在于：你无锁地读着，别人把底层数组 resize 掉怎么办？答案是 QSBR（quiescent-state based reclamation，静默期回收）：旧内存不立即释放，等到所有线程都经过一次“静默点”（确定没人还握着旧指针）才回收。读者零开销，代价是内存释放略延迟。

> **黑卡总结的修辞：这套“鸡尾酒”（cocktail）就是 30 年工程演化的浓缩——1996 年只有锁，2016 年只有原子，2021+ 是四种机制各司其职。**

第 15 页 各库的重管道工程 · Every big library re-did its object plumbing



这页在讲什么：解释器给了新地基（第 14 页），每个库还得自己重做水电——这页是“现在各库怎么适配”的总表。

CPython C API 新原语

`PyMutex`：3.13 新增的 1 字节轻量锁，可静态初始化；关键特性是它挂进了运行时——线程在 `PyMutex` 上阻塞时会自动释放 GIL/配合 GC，从设计上杜绝“我的锁和 GIL 互等”的死锁。这就是官方推荐所有扩展用它换掉自带锁的原因。加上 critical sections 和 `*_GetItemRef`（第 13、14 页），构成扩展作者的新工具箱。

NumPy：审计每个 C 全局变量

NumPy 的 C 代码里散落着大量 static 全局：配置、状态、PyObject 缓存。FT 下这些全是竞争点（缓存被两个线程同时覆写 → 内存损坏）。工作 = 逐个排查 + 把所有 `PyThread_type_lock` 移植到 `PyMutex`。出处：Quansight 的 free-threading 系列博客。

PyTorch vs TensorFlow（呼应第 10 页）

PyTorch 审计了 ATen（张量库）和 autograd 的线程安全，2.9 出预览、2.10（2026-01）起正式发 cp314t wheel。TensorFlow 至今没动——它的并行本来就在 Python 之下（第 10 页逃生舱②），用户感知不到 GIL，适配动力最小。对比点睛：当年绕得越彻底，今天越不着急。

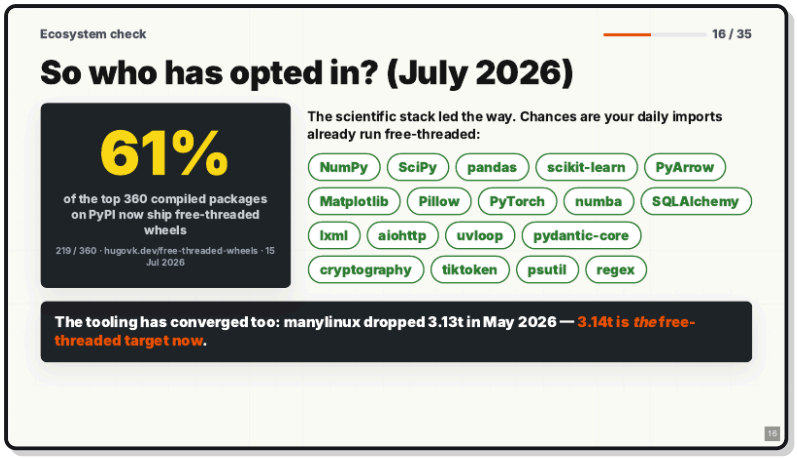
Cython / PyO3

Cython 3.1+：加一行 `freethreading_compatible=True` 指令即可声明兼容（生成的胶水代码自动处理），热点自己用 `cython.pymutex`。PyO3（Rust 绑定）最优雅：Rust 编译器的 `Send` / `Sync` 类型系统在编译期拒绝数据竞争——不安全的代码根本编不过。

黄卡（新措辞）

“Not app code — plumbing, done once per library. 99% of Python users never see it.” 给应用开发者吃定心丸：这些苦活是库作者一次性做完的。

第 16 页 生态大盘 · So who has opted in? (July 2026)



这页在讲什么：适配不是空话——用一个硬指标量化生态进度：PyPI 下载量前 360 的编译型包里，61%（219/360）已发布 free-threaded wheel。

数字出处与口径

hugovk.dev/free-threaded-wheels（Hugo van Kemenade 维护的追踪站，同款方法论的“pythonwheels.com”变体）。口径：只统计带扩展模块的包（纯 Python 包天然兼容，不用统计）；“支持”= 在 PyPI 正式发布了 t 标签 wheel。数据抓取日期 2026-07-15（讲前一天可再刷新一次）。

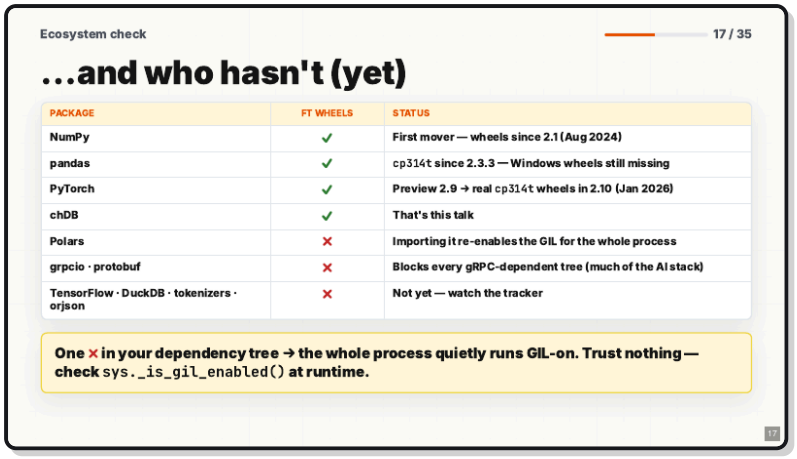
绿色 pill 墙的挑选逻辑

全是听众日常 import 的名字：科学栈（NumPy/SciPy/pandas/scikit-learn/PyArrow/Matplotlib）、AI（PyTorch/tiktoken/numba）、Web/基础设施（SQLAlchemy/lxml/aiohttp/uvloop/pydantic-core/cryptography）。信息：“你的日常技术栈大概率已经就绪”。

manylinux dropped 3.13t

manylinux 是官方 Linux wheel 构建基础镜像。2026 年 5 月它移除了 3.13t（实验性构建，已被转正的 3.14t 取代）——生态的目标版本收敛到 3.14t，这是“工具链也定型了”的信号。出处：pytorch/pytorch#182949 里引用的 manylinux 变更。

第 17 页 谁还没上车 · ...and who hasn't (yet)



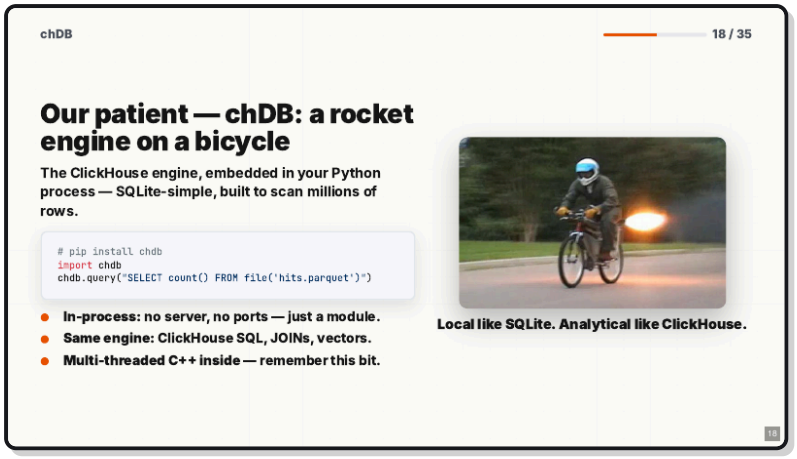
这页在讲什么：61% 的另一面。表格逐行：

表格逐行

- **NumPy ✓**：第一个吃螃蟹——2.1（2024-08）就发了 FT wheel，全生态的破冰者。
- **pandas ✓**：2.3.3 起有 cp314t（macOS/Linux；Windows 的 FT wheel 还没有——细节出处 pandas-dev/pandas#63649）。
- **PyTorch ✓**：2.9 预览 → 2.10（2026-01）正式。
- **chDB ✓**：“That's this talk”——自我指涉的小幽默，顺便立住 chDB 的先进性。
- **Polars ✗**：没发 FT wheel 且未声明兼容 → `import polars` 的瞬间整个进程的 GIL 被重新打开（第 12 页机制①的活例子）。社区有人自编 wheel，官方还没跟。
- **grpcio · protobuf ✗**：这俩是无数包的底层依赖（Google 系、几乎所有 AI SDK）。它们不发 FT wheel，上面整棵依赖树都动不了——“基础包卡脖子”现象（grpc/grpc#41461 是催更 issue）。
- **TensorFlow · DuckDB · tokenizers · orjson ✗**：知名度高但还没上车的代表。DuckDB 顺带是 chDB 的直接竞品，这行有个不用说破的对比。

> **黄卡 = 本页可带走的操作项：依赖树里任何一个 ✗ 都会让全进程静默回到 GIL。别信配置，跑起来查 `sys._is_gil_enabled()`。**

第 18 页 病人入场：chDB · Our patient — chDB



这页在讲什么：案例主角登场。比喻链：ClickHouse 引擎（火箭发动机）装进你的 Python 进程（自行车）——“rocket engine on a bicycle”。

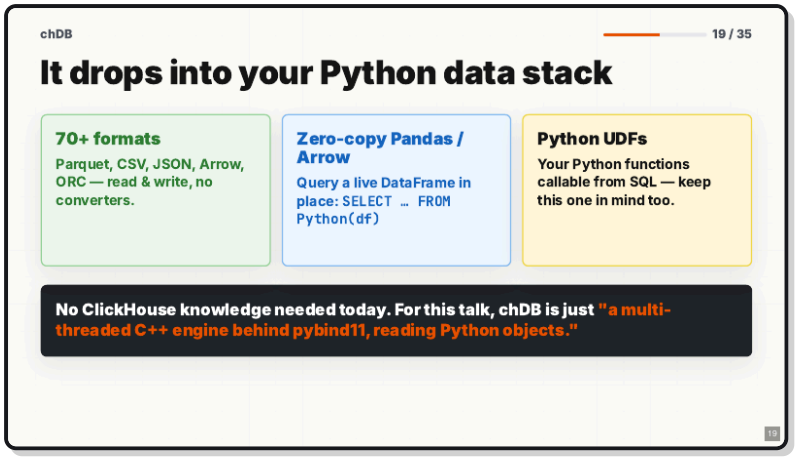
in-process（进程内）

chDB 不是数据库服务器：没有独立进程、端口、连接。`pip install chdb` 后它就是个模块，SQL 引擎跑在你的 Python 进程地址空间里。类比：SQLite 之于 MySQL = chDB 之于 ClickHouse。

“Multi-threaded C++ inside — remember this bit”

刻意埋的伏笔：ClickHouse 引擎天生多线程（一个查询内部就会开满核）。这让 chDB 成为绝佳测试场——引擎侧早就并行，唯一的串行点恰好就是 GIL 边界（第 20 页收网）。

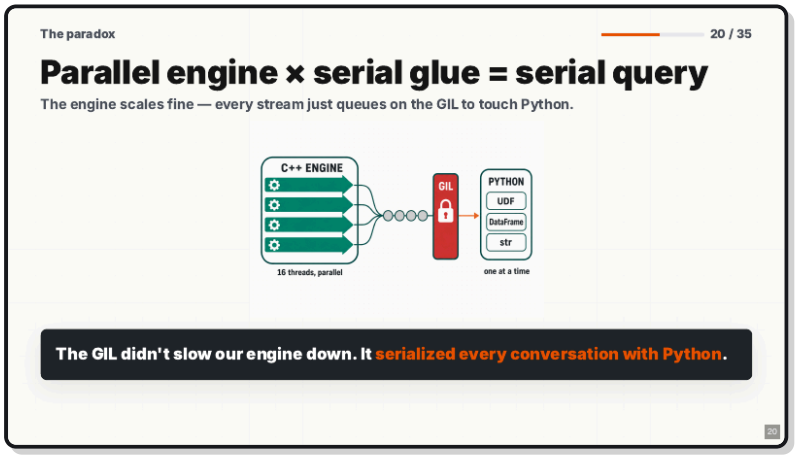
第 19 页 chDB 与数据栈 · It drops into your Python data stack



这页在讲什么：chDB 的三个能力卡，同时又都是后文的伏笔：70+ 格式（Parquet/CSV/JSON/Arrow 直读直写）；零拷贝查询 DataFrame（`SELECT ... FROM Python(df)`，直接扫 pandas 内存，不导出不复制——第 28 页 Arrow 路径的基础）；Python UDF（SQL 里调用你的 Python 函数——第 22–24 页的主线）。

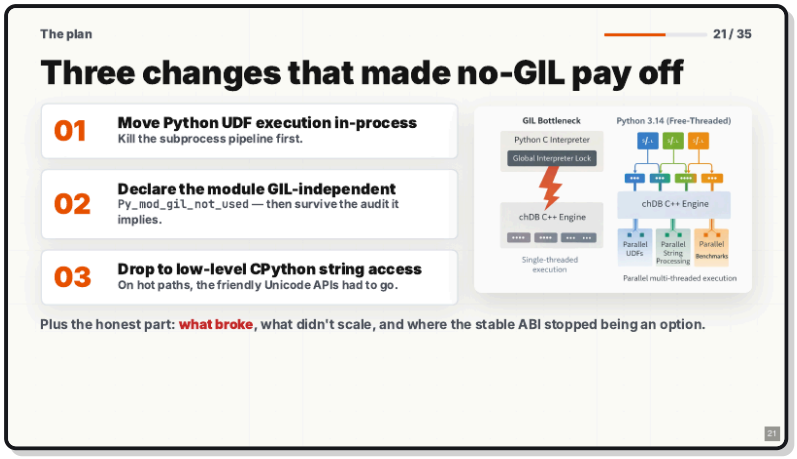
> **黑卡明确对观众的知识要求：“今天不需要懂 ClickHouse，把 chDB 当成‘pybind11 后面一个要读 Python 对象的多线程 C++ 引擎’即可。”（pybind11：主流的 C++/Python 绑定库。）**

第 20 页 悖论 · Parallel engine × serial glue = serial query



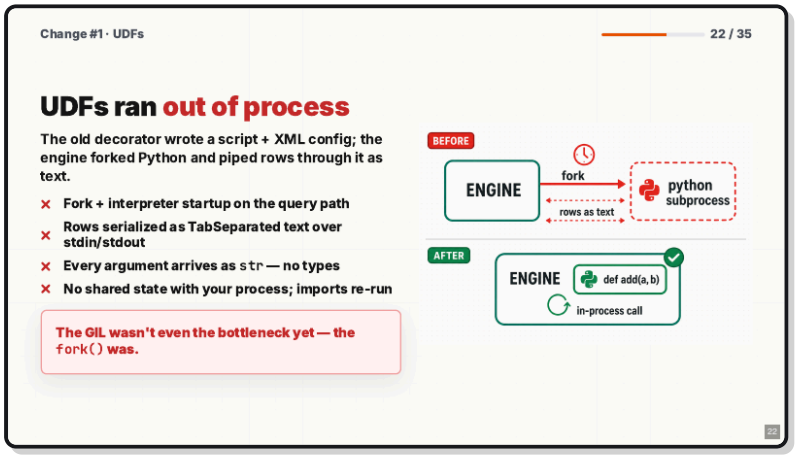
这页在讲什么：第 11 页那张“收费闸”总图的 chDB 特写：16 条引擎流水线（stream）并行扫数据，但每条流一旦要碰 Python——调 UDF、读 DataFrame 列、转字符串——就在 GIL 前排成单队。punch line 值得背下来：“GIL 没有拖慢我们的引擎，它把引擎和 Python 的每一次对话变成了串行。”

第 21 页 作战计划 · Three changes that made no-GIL pay off



这页在讲什么：目录页。移植 = 三步走：① UDF 从子进程搬进进程内（先拆架构性串行）；② 声明模块 GIL-independent 并通过随之而来的审计；③ 热路径改用底层 CPython 字符串访问。次序有讲究（第 24 页黑卡会点破：先进程内、再摘 GIL，顺序反了白干）。“Plus the honest part”预告第 29 页的翻车记录——给 talk 定下“不吹牛”的调性。

第 22 页 改造一 · 旧 UDF · UDFs ran out of process



这页在讲什么：历史包袱：ClickHouse 服务器的 UDF 机制是“外部可执行脚本”，chDB 直接继承了它——每次查询 fork 一个新 Python 解释器，数据按 TabSeparated 文本走 stdin/stdout 管道。

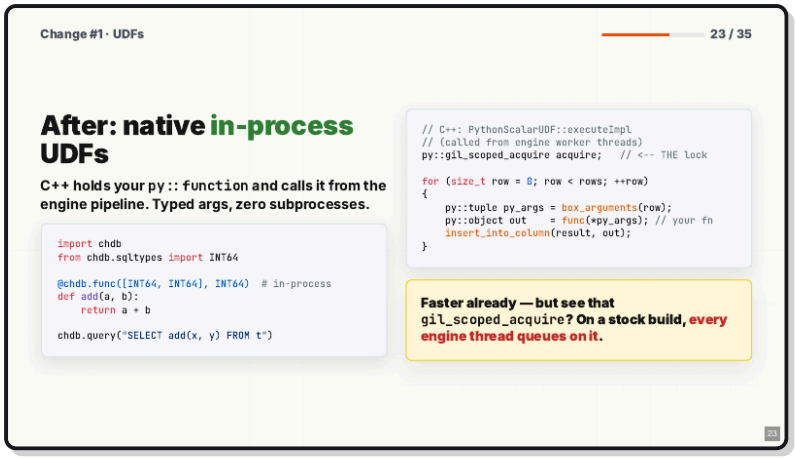
四个 ✗ 逐条

① fork + 解释器启动开销直接算在查询延迟里；② 行数据序列化成文本再解析回来（两头浪费）；③ 所有参数到达时都是 `str`——类型信息全丢，用户函数里还得手动转；④ 子进程和主进程零共享——你在主进程里 import 的模块、建的连接，UDF 里全都要重来。

红卡的自嘲

“GIL 还没轮到出场，fork 就已经是瓶颈了”——先承认旧架构烂在更基础的层面，为“先修架构再谈 no-GIL”的方法论铺路。

第 23 页 改造一 · 新 UDF · After: native in-process UDFs

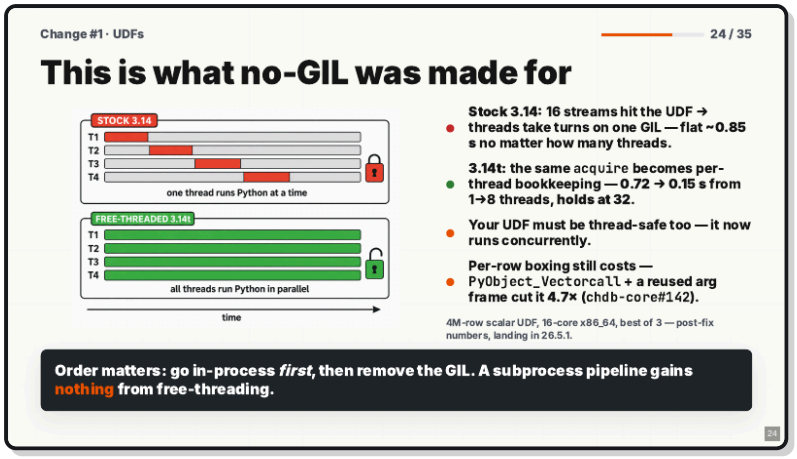


这页在讲什么：新机制：C++ 侧持有你的函数对象（`py::function`），引擎流水线里直接调用。装饰器 `@chdb.func([INT64, INT64], INT64)` 声明参数/返回类型 → 引擎按类型直接装箱，不再有文本管道，不再有子进程。

右侧 C++ 代码里的 py::gil_scoped_acquire

pybind11 的 RAII 锁：引擎工作线程（非 Python 线程）要调 Python 函数前，必须先获取 GIL。黄卡点出问题：在普通构建上，这一行让 16 条引擎线程在同一把全局锁上排队——进程内化解决了 fork 税，但 GIL 税原封不动。悬念交给下一页。

第 24 页　改造一 · 收网 · This is what no-GIL was made for　本版更新



这页在讲什么：同一份代码，两种世界（左图时间线）：stock 3.14 上 16 线程轮流持锁 ≈ 单核吞吐；3.14t 上同一个 `gil_scoped_acquire` 退化成每线程的簿记操作，16 条流真并行地跑你的 UDF。v3 起两条对比 bullet 各自带上了实测数字——时间线示意图第一次有了数字背书。

新数字从哪来（chdb-core#142 修复后，2026-07-23 合并）

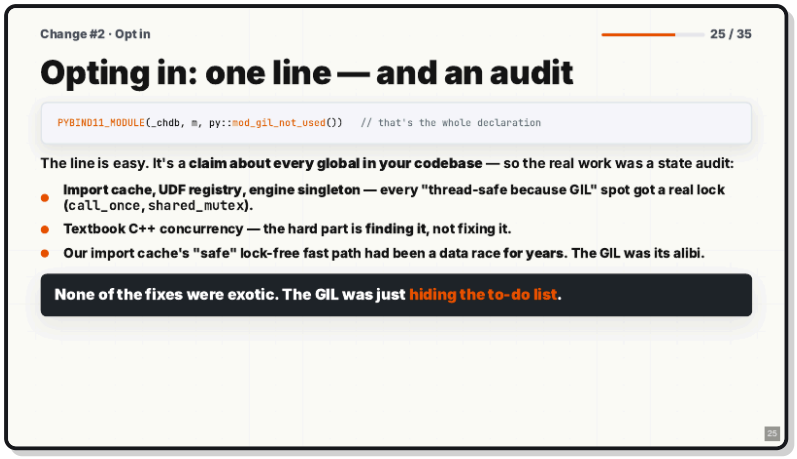
基准：`fadd((a*31+b)%97)` 扫 400 万行 `numbers_mt`，16 核 x86_64，best of 3。**stock 3.14**：单线程 0.83 s，加到 8/32 线程仍是 ~0.85 s——GIL 下平的，如预期，这就是 bullet ① 的 “flat no matter how many threads”。**3.14t**：1 线程 0.72 s → 8 线程 0.15 s，32 线程 0.17 s——扩展到满核后不再退化（#128 修复的直接证据）。两个隐藏亮点讲的时候可以点：FT 单线程从修复前比 stock 慢 ~1.5× 变成**略快**；同一批 hoisting 优化让 stock 构建也白拿了 ~2.8×（2.31 → 0.83 s）。

两条附加的诚实声明（第②条已升级）

① 你的 UDF 现在真的会被并发调用——用户函数自己也得线程安全（改全局变量的 UDF 会翻车）；② no-GIL 消灭的是争用（contention），不是调用开销——per-row boxing 的成本还在，但 #142 用 `PyObject_Vectorcall` + 复用参数帧（每行不再新建 tuple）把它砍了 4.7×。概念不变，数字变了。

> **黑卡：顺序定律。“先进程内，再摘 GIL。子进程管道从 free-threading 里得到的收益是零。”——因为子进程各有各的 GIL，摘不摘无感。这是给所有想移植的人的第一条方法论。**

第 25 页 改造二 · 声明与审计 · Opting in: one line — and an audit



这页在讲什么： `PYBIND11_MODULE(_chdb, m, py::mod_gil_not_used())` ——申报“本模块无 GIL 也安全”就这一行（pybind11 ⩾ 2.13 支持；裸 C API 用 `Py_mod_gil` slot）。但这行是一份关于你代码库里每一个全局状态的宣誓，所以真正的工作是状态审计：

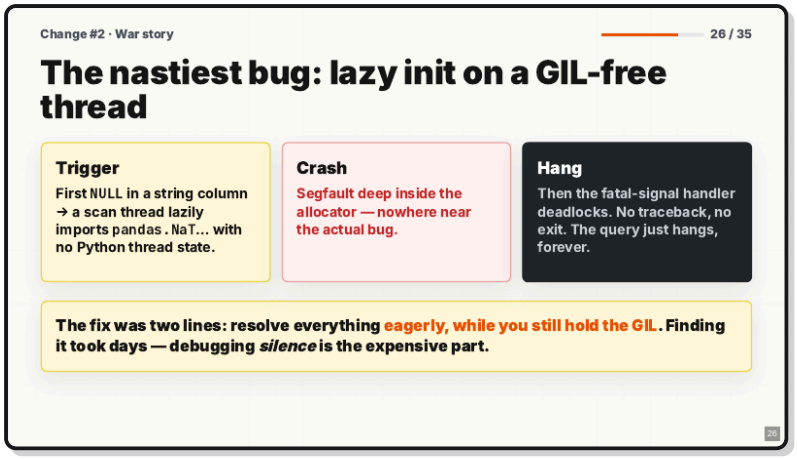
审计清单

import 缓存、UDF 注册表、引擎单例——所有“靠 GIL 保平安”的点补真锁（`std::call_once`、`std::shared_mutex`）。修法都是教科书级 C++ 并发，难点在找全，不在修。

最扎心的发现

import 缓存那条“无锁快路径”多年来一直是数据竞争——GIL 是它的不在场证明（alibi），恰好从没两个线程同时跑到那里。黑卡收束：“这些修复毫不新奇，GIL 只是一直把待办清单藏着。”

第 26 页 战地故事 · The nastiest bug: lazy init on a GIL-free thread



这页在讲什么：移植期间最难缠的一个 bug，三幕剧：

Trigger（触发）

字符串列里出现第一个 NULL → 扫描线程需要拿 `pandas.NaT`（pandas 的“缺失时间”单例）做比较 → 触发懒加载 import——而这个扫描线程是纯 C++ 线程，没有 PyThreadState（第 4 页埋的伏笔），没有任何执行 Python 代码的资格。

Crash（崩溃在错误的地方）

段错误发生在 `_PyObject_Malloc`——分配器深处，离真正的病灶（那次懒 import）十万八千里。这类“崩在 A、病在 B”是无 GIL 调试的常态。

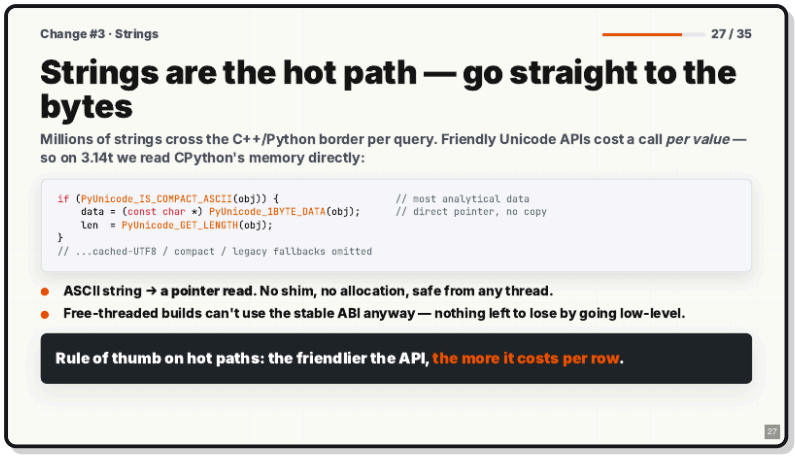
Hang（雪上加霜）

崩溃信号处理器自己又死锁——没有 traceback、没有退出码，查询永远挂着。调试“沉默”比调试崩溃贵得多（黄卡的教训）。

修复

两行代码：在还持有 GIL 的准备阶段，把扫描线程将来要用的对象（NaT、NA）提前解析好。通用规则：凡是可能被非 Python 线程触到的懒初始化，一律改成“持 GIL 时提前做”。

第 27 页 改造三 · 字符串 · Strings are the hot path — go straight to the bytes



这页在讲什么：分析型查询 = 每秒几百万字符串跨越 C++/Python 边界。友好 API（如 `PyUnicode_AsUTF8AndSize`）每个值一次函数调用，热路径付不起——3.14t 构建直接读 CPython 字符串的内部内存。

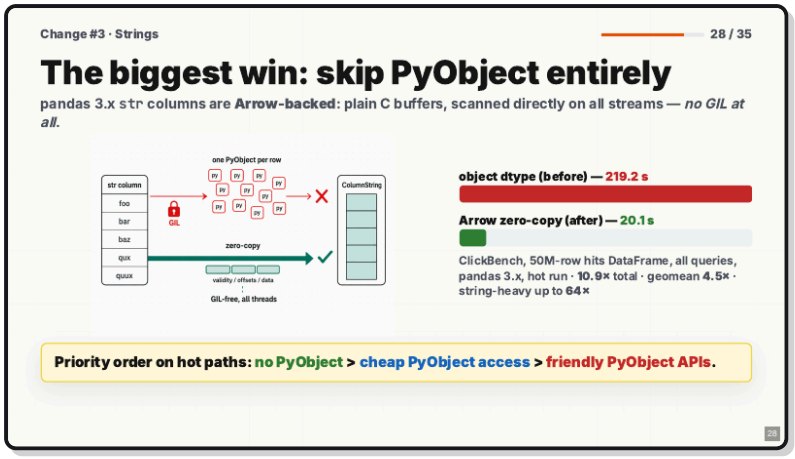
代码块：PyUnicode_IS_COMPACT_ASCII 路径

CPython 字符串内部有多种存储形态。纯 ASCII 的“紧凑”字符串（分析型数据的大多数），字符数据就躺在对象头后面——`PyUnicode_1BYTE_DATA` 拿到裸指针，一次指针读，零拷贝、零分配，任何线程都安全（字符串不可变）。非 ASCII 再走缓存 UTF-8 / kind 1-2-4 字节等 fallback（代码里省略）。

“FT 构建反正用不了 stable ABI”

普通构建为了 abi3 兼容性不能碰这些内部宏（chDB 曾为此维护一个按版本编译的 shim 动态库，每个字符串一次跨 .so 调用）；FT 构建没有 abi3 可失去 → 索性全上底层 API。黑卡的经验法则：热路径上，API 越友好，每行的税越重。

第 28 页　最大的胜利 · Skip PyObject entirely



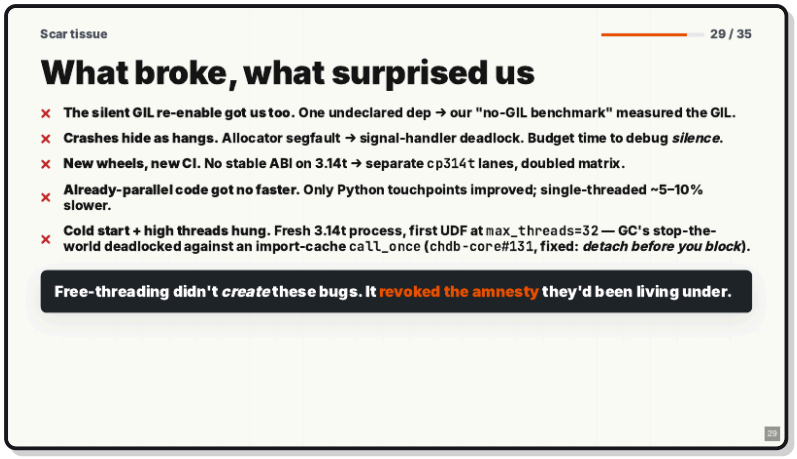
这页在讲什么：比“便宜地访问 PyObject”更狠的是“根本不经过 PyObject”。pandas 3.x 的字符串列默认 Arrow-backed：底层是 Apache Arrow 的三段裸 C 缓冲区（validity 位图 / offsets 偏移 / data 字节），没有每行一个的 Python 对象。chDB 直接把这三段缓冲区零拷贝喂给所有引擎线程——这条路完全不涉及 GIL，在普通 3.14 上也全速。

基准数字（右侧条形图）——deck 里 ClickBench 数字现在只在这一页

ClickBench（业界标准分析基准）的 5000 万行 hits 表装成 DataFrame，跑全部查询：object dtype 老路 219.2 秒 → Arrow 零拷贝 20.1 秒 = 10.9×；几何平均 4.5×；字符串最重的查询最高 64×。注意归因：这组数字是**零拷贝**的功劳，和 no-GIL 无关（重构后的第 30 页因此不再引用它们，避免观众把 10.9× 记到 free-threading 头上）。

> **黄卡：热路径优先级公式。不碰 PyObject > 便宜地碰 > 友好地碰。这页和第 31 页共同构成 talk 最重要的工程结论：能用数据布局解决的，别用锁解决。**

第 29 页　伤疤 · What broke, what surprised us　本版更新



这页在讲什么：翻车合集（诚实环节），现在是五条：

①–④（原有四条）

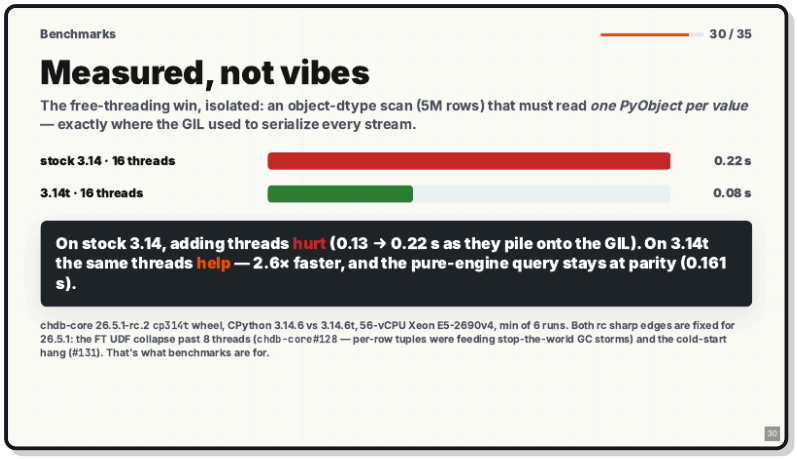
① 自己也中了“静默 GIL 重启”——某个依赖没声明，跑了一阵“no-GIL 基准”其实测的是 GIL（所以要在进程内部断言 `sys._is_gil_enabled()`）；② 崩溃伪装成挂起（第 26 页展开过）；③ 告别 stable ABI = CI 矩阵翻倍、pybind11 需要 vendor（自带修改版）；④ no-GIL 没有加速本来就并行的 C++ 部分——只有 Python 触点变快，而单线程整体还慢了 5–10%。

⑤ Cold start + high threads hung（chdb-core#131——已修复，slide 已改过去时）

现象：全新的 3.14t 进程里，**第一条**查询就是 `max_threads=32` 的 UDF 查询 → 无限等待，永不返回（当时的规避法：先跑一条小预热查询）。现在有了确诊（PR #141）：**GC 的 stop-the-world 和 import cache 的 `std::call_once` 死锁**——一条线程等 call_once 时仍保持 attached（持有解释器线程状态），而 FT 的 GC stop-the-world 要求所有 attached 线程先停下，call_once 的持有者又被 GC 挡住……循环等待。触发它的高频 STW 风暴正是 #128 的每行 tuple 分配（第 30 页详述），所以两个 bug 是同一场风暴的两张面孔。修复 = 等待前先 detach，slide 上的新教训是“**detach before you block**”——和第 26 页“持 GIL 时提前做”同族：都是『阻塞点 × 解释器状态』的时序规则。讲的时候可以强调：这一条就是靠第 30 页那套基准测出来的（benchmark 的第二价值：抓 bug）。

> **黑卡金句（全 talk 的 thesis）：“free-threading 没有制造这些 bug，它只是吊销了它们的赦免令（amnesty）。”**

第 30 页 基准 · Measured, not vibes 整页重构



这页在讲什么：重构后只讲一个故事：把 free-threading 的收益单独隔离出来。载体是 object-dtype 扫描——500 万行 `dtype=object` 字符串列，`Python(df)` + LIKE 查询，每个值都必须读一次 PyObject，正好落在 GIL 过去串行化每条流的位置。两根条形：stock 3.14 · 16 线程 = 0.22 s（红），3.14t · 16 线程 = 0.08 s（绿）。

为什么砍掉旧版的双卡片

旧版左卡放 ClickBench 10.9×/64×，右卡放 no-GIL 对照——两组数字归因不同（前者是零拷贝的功劳，后者才是 free-threading 的），放在一页容易让观众把 10.9× 记到 no-GIL 头上。重构后：ClickBench 留在第 28 页（零拷贝的战果），这一页只回答“no-GIL 本身值多少”。一页一个论点，归因干净。

黑卡：方向相反才是重点

“On stock 3.14, adding threads hurt (0.13 → 0.22 s)。On 3.14t the same threads help — 2.6× faster。”最有教学价值的不是 2.6× 本身，而是方向：stock 上从 1 线程加到 16 线程反而从 0.13 s 恶化到 0.22 s（全部线程堆在 GIL 上，争用净损耗）；3.14t 上同样的加线程改善到 0.08 s。同一份代码、同一台机器，唯一变量是有没有 GIL。黑卡还带一句对照组：纯引擎查询（5000 万行 cityHash 聚合，不碰 Python 对象）0.161 s vs 0.161 s——C++ 引擎本来就不持 GIL，free-threading 对它既无收益也无损耗，印证第 31 页“你的 C++ 已经释放 GIL 的部分不会更快”。

脚注：测量口径（被问到要能答）

单一硬件、单一口径：chdb-core 26.5.1-rc.2 的真 `cp314t` wheel；CPython 3.14.6 vs 3.14.6t，两个解释器来自 python-build-standalone 同一 release（20260623）、同编译工具链，唯一变量就是 GIL；机器是 VAST 上的 56 vCPU Xeon E5-2690v4；每项取 6 轮最小值（共享宿主机，最小值≈无干扰）。旧版里 Apple M4 Pro 的那组数字已从 deck 中移除——单硬件叙事更干净。

脚注：两个 rc sharp edges——都已修复（PR #141/#142，2026-07-23 合并） 本版更新

脚注文字已改为“Both rc sharp edges are fixed for 26.5.1”，并把被上一版砍掉的金句请了回来：“That's what benchmarks are for.”

#128 的确诊（perf profile，不再是猜）：热路径每行做了三件蠢事——重新推导循环不变量（Const/Nullable 解包、类型检查、经由 `Field` 的 dtype 分发）、经 pybind11 每行构造一个 **gc-tracked** `py::tuple`、结果侧 `GetPythonObjectType` 先查 import 缓存再查内建类型。FT 上那个每行 tuple 主导了 GC 触发率，而 FT 的 young GC 是 **stop-the-world**（暂停所有查询线程）：线程越多 → 分配率越高 → STW 越频繁——这就是“超过 8 线程负扩展”的全部秘密。修复：不变量 hoist 成 per-block spec、常见类型走 `IColumn` 直取；FT 用 `PyObject_Vectorcall` + 复用参数帧（每行零 tuple）；abi3 构建没有 vectorcall（不在 3.9 stable ABI 里），退而直接 `PyObject_CallObject` 跳过 pybind11 的 unpacking 收集器。修复后数字见第 24 页。

被追问“你们当初的三个假设呢”时的答法：issue 里的三个假设（biased refcounting 计数器乒乓 / UDF registry 锁 / pybind11 内部锁）**没有一个是主因**——真凶是 GC STW 风暴。这本身就是个好故事：FT 的性能直觉还没建立，猜是猜不准的，profile 才算数——正好又落回本页标题“Measured, not vibes”。#131 见第 29 页注释。

数字的可信度声明

FT 侧 `import chdb` 后 `sys._is_gil_enabled()` 为 False（验证过没被静默重启）。cp314t wheel 来自 v26.5.1-rc.2 GitHub release（正式 26.5.1 才上 PyPI）；chdb 仓库的 FR #505 已于 2026-07-16 回复并关闭。

第 31 页 决策指南 · When does no-GIL actually help?



这页在讲什么：给观众的“什么时候该上”对照表。**有效**：多线程要执行 Python（UDF/回调）、多线程要读 PyObject（object 列）、且 GIL 是测量出来的瓶颈（不是感觉）。**无效**：你的 C++ 早就释放 GIL 干重活（第 10 页逃生舱①已经够用）、能彻底绕开 Python 对象（Arrow/buffer/NumPy——第 28 页刚证明这条路更快）、单线程负载（白付 5–10%）。

> **黑卡自曝：“我们移植里最大的单项收益来自零拷贝而非摘锁——free-threading 修的是零拷贝够不着的那部分：真并行地跑 Python 本身。”**

第 32 页 观众行动页 · You don't need to port anything. Try it this week



这页在讲什么：全 talk 的落地页，写给普通用户（不是扩展维护者）：三条命令今天就能试。

三步走（左侧代码块）

① `uv python install 3.14t`——uv 直接管理 free-threaded 解释器，一分钟装好（不用 uv 的话 python.org 安装器也带 t 选项）；② 原封不动跑自己的测试/应用，盯着 GIL 重启的 RuntimeWarning；③ 信任但要验证：import 你的应用后打 `sys._is_gil_enabled()`，False 才是真的在无 GIL 下跑（第 12/17 页的静默重启机制随时可能骗到你）。

黄卡：为什么用户“零改动”

`ThreadPoolExecutor` / `threading` 的 API 语义完全不变——过去这些代码在 CPU 型任务上“假并行”，在 3.14t 上同一份代码直接开始真扩展（第 30 页 objscan “加线程从有害变有益”的对照就是证据）。

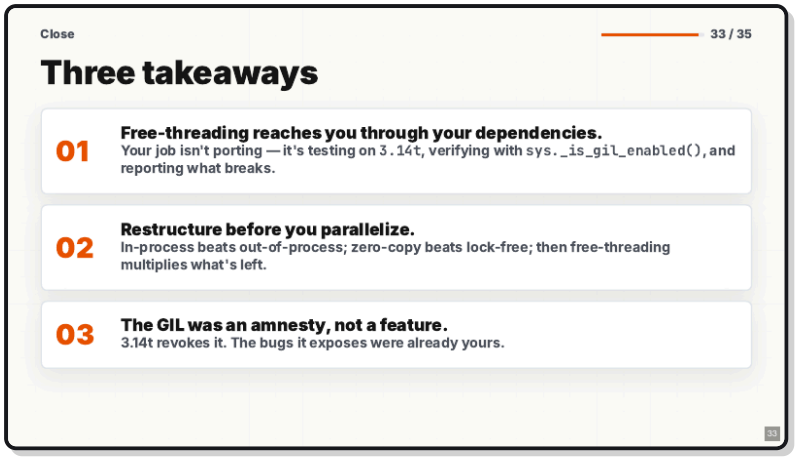
右卡：普通用户帮社区的三种方式

① 在 3.14t 上跑自己的测试套件并报 bug——真实世界的破坏案例是维护者们的第一需求（出处：Quansight/py-free-threading 的社区总结）；② 给挡路的依赖催 wheel（去点赞/开 issue，比如 grpcio 那类第 17 页 ✗ 名单）；③ 发布你的加速数字——“我的负载快了 N 倍”是推动库排期的最强论据。

维护者内容去哪了

压缩成底部一行：porting checklist 在 py-free-threading.github.io——“本 talk 做的一切的通用版”。（原来的五步清单页已删除——听众里维护者是极少数。）

第 33–35 页 收尾三连 · Takeaways · Thanks · Q&A



Takeaways 三条 = 全 talk 压缩包（第一条是用户视角）：① free-threading 是通过你的依赖到达你的——你的任务不是移植，是在 3.14t 上测试、用 `sys._is_gil_enabled()` 验证、把坏掉的报上去；② 先重构再并行（进程内 > 跨进程；零拷贝 > 免锁；free-threading 放大剩下的部分）；③ GIL 是赦免令不是功能——3.14t 吊销它，暴露的 bug 本来就在那里。

Thanks 页（标题“Take 3.14t for a spin”）

行动号召：`pip install chdb` 试引擎、`python3.14t your_tests.py` 试未来。资源指路：py-free-threading.github.io、PEP 703/779 原文、chDB 文档。

Q&A 页

大字号 + 双 logo，留给提问环节当背景板。

附录 · 可能被问到的问题 Anticipated Q&A

Q：subinterpreters（子解释器，PEP 684）不是也能并行吗？和 free-threading 什么关系？

子解释器 = 一个进程里多个隔离的解释器，各有各的 GIL——并行可以，但解释器之间不能直接共享 Python 对象（要走通道序列化），更像“轻量版 multiprocessing”。free-threading 是真共享内存的并行。两者官方都在推进，解决的问题不同。

Q：单线程慢 5–10%，凭什么值？

看负载：单线程程序确实不该换（第 31 页右卡第三条）。但一旦并行度 ≥ 2，5–10% 的固定税远小于 N 倍的并行收益。且这个税逐版本在降（3.13 约 40% → 3.14 约 5–10%）。

Q：什么时候变成默认?

没有官方日期。PEP 703 的 Phase III 有明确条件：生态就绪 + 性能差距足够小，由指导委员会拍板。可以引用第 16 页：61% 且 manylinux 已收敛到 3.14t——趋势明确，时间未定。

Q：free-threading 下还需要 asyncio 吗?

需要。asyncio 解决的是“等待很多 I/O 时不浪费线程”（并发），free-threading 解决“CPU 并行”。两者正交，可组合（多个事件循环跑在多条线程上是新解锁的玩法）。

Q：GIL 移除后引用计数还准吗? weakref、gc 呢?

准——biased refcounting 在对象销毁时合并两个计数器，语义不变。循环 GC 改成 stop-the-world（短暂暂停所有线程），weakref 等机制都做了线程安全化。这些属于第 14 页“黑魔法”的深水区，被问到再展开。

Q：你们 rc 阶段的 #128 / #131 修好了吗? 根因是什么? 已重写

都修好了（PR #141/#142，2026-07-23 合并，进 26.5.1 正式版）。**#128**（UDF 超 8 线程负扩展）根因：每行经 pybind11 新建一个 gc-tracked tuple → FT 的 young GC 是 stop-the-world，分配率随线程数升高 → STW 风暴，线程越多暂停越频繁。修复 = `PyObject_Vectorcall` + 复用参数帧（每行零分配）+ 循环不变量 hoist；修复后 0.72 → 0.15 s（1→8 线程）、32 线程不退化，stock 构建也顺带快了 2.8×。**#131**（冷启动挂死）根因：线程 attached 状态下等 import-cache 的 `call_once`，和 GC stop-the-world 循环等待；修复 = 阻塞前先 detach。注意：#128 issue 里原先的三个假设全不是主因——profile 才给出真凶。回答姿态：这正是“Measured, not vibes”——基准不但量化收益，还抓出并确诊了两个真 bug。

主要参考：PEP 703/779/683 原文 · py-free-threading.github.io（移植指南与兼容追踪）· Victor Stinner “Free Threading internals” 系列 · LWN “Progress on the Gilectomy” · Quansight free-threading 年度回顾 · hugovk.dev/free-threaded-wheels · chdb-core#128 / #131 · chDB 移植工作的第一手材料（本 deck 前身）。